Bags, Vectors & Transformers

Information Extraction with an LLM · Bonus, Part 15

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

From Labels to Records

	Classification	**Extraction**
Output per text	one label	a list of **records**, each with several fields
Example	the policy area of a bill	every quote: speaker, role, party, words
The codebook	label definitions	a **schema**: fields, allowed values, rules
Validation	accuracy per label	per record (found? invented?) and per field

In social science: actors and the claims they make, protest events (who, where, when, how many), relations ("who criticizes whom"), the promises in a manifesto, and the voices in the news.

The Schema Is Your Codebook

```
Role = Literal["politician",
"government official",
"expert or academic", "business", ...]
Party = Literal["Labour", "Conservative",
"Liberal Democrat", "other party",
"no party"]

class Quote(BaseModel):
speaker:  str
role:  Role
party:  Party
quote:  str   # copied exactly

class ArticleQuotes(BaseModel):
quotes:  list[Quote]
```

A **Pydantic** class turns the codebook into code:

- `Literal` fields allow only the listed values
- a list allows any number of records, including none
- the same class **checks** every answer

The decision rules (what counts as a quote, what to skip) stay in the instructions, as they would in a codebook for human coders.

Why Not Just Ask for JSON?

"Reply with a JSON list" works most of the time. The rest:

- code fences or a sentence around the JSON
- `{"quotes": [...]}` instead of a list
- renamed keys: `name` instead of `speaker`
- values outside the codebook: "Chancellor", "Tory"

Each needs its own cleaning rule.

Structured outputs. Ollama takes the schema and, while the model writes, simply doesn't allow any token that would break it. The answer always parses and always uses allowed values.

```
ollama.chat(model=MODEL,
messages=messages,
format=ArticleQuotes.model_json_schema(),
options={"temperature": 0})
```

It fixes the **format**, not the **content**: wrong speakers and missed quotes still happen.

Is It Really in the Text?

We asked for **exact words**, so every quote can be checked automatically:

exact	word for word in the article
partly	a long stretch matches ($\geq 60\%$)
not in text	neither

Plus: does the speaker's name appear in the article at all?

What models change: quotes stitched together across "he said", grammar and punctuation "tidied up", words dropped from the middle, indirect speech turned into a direct quote.

Decide **before** looking at results: keep exact, check or repair partly, drop the rest. Report how many each rule affected.

Did It Find Everything? Is Every Field Right?

Recall against a pattern

Passages of five or more words in quotation marks are mostly speech. A regular expression finds them in every article; a passage counts as found if one of the model's quotes contains it.

A rough standard: some quoted passages are titles or slogans, so 100% isn't the target. Read the misses.

Fields by hand

No pattern can tell who is speaking. Check a random sample of quotes against the article: right speaker? right party?

Typical errors: a quote that continues in the next paragraph, two people named close to the quote, one politician quoting another.

Who Gets a Voice?

BBC political news, 2004–2005 (Greene & Cunningham, 2006): the run-up to the 2005 UK general election. Qwen2.5 7B through **Ollama**, on Colab's free GPU.

Each result goes to a cache file as soon as it's done, so a disconnected session continues where it stopped (Bonus 13).

From records to findings: quotes by role, quotes by party, the most quoted people. A classic question in journalism research, now at the scale of a whole corpus.

Merge spelling variants first ("Mr Brown", "Gordon Brown", "the chancellor"), and validate on a hand-coded sample before you count.

In the Notebook

- BBC political news, paragraphs restored, quoted passages found with a pattern
- Qwen2.5 7B running locally in Ollama
- Free-form JSON versus structured outputs with a Pydantic schema
- A cached, resumable extraction run
- Grounding checks, recall, and checking speakers by hand
- Who gets quoted; adding a field; a smaller model compared

`bonus/notebooks/15_information_extraction_with_llms_exercises.ipynb` · 5 exercises · GPU